INTERNET APPENDIX

Sequential Completion in Dependent Panels

Giovanni Urga and Emilio Zanetti Chini

This Internet Appendix documents the empirical and simulation material that is useful for replication and auditability but not needed for the main line of the paper. The main text states the econometric framework, the formal approximation results, and the headline Monte Carlo and empirical findings. The appendix records the implementation details that a reader would need to reconstruct the exercises: the real-time data layer, the frontier-window construction, the fixed method classes, release-validation diagnostics, target-object timing, bootstrap inference for prespecified empirical comparisons, and the extended simulation output.

All exhibits follow the same convention. The title above each exhibit is intentionally short. The note below the exhibit is longer and states the sample, timing convention, loss object, and interpretation. This keeps the appendix readable while making each table or figure self-contained. The appendix is therefore not a second narrative version of the paper; it is an audit file for the timing, validation, and simulation objects used in the main text.

The appendix is organized as follows. Section 1 documents the real-time FRED-MD data layer and the vintage-index timing convention. Section 2 reports frontier-window diagnostics, showing how the release-validation sample changes when the current-frontier window is widened. Section 3 records the fixed method classes and their real-time feasibility restrictions. Section 4 reports release-validation state diagnostics, state risks, and selector behavior. Section 5 documents the downstream target-object design and the relation between local release loss and INDPRO target-object loss. Section 6 describes bootstrap inference for prespecified empirical comparisons. Section 7 collects the extended Monte Carlo exhibits, including calibration, bridge diagnostics, scaling exercises, application-size regimes, and bridge-strength checks.

1 Real-Time FRED-MD Data Layer

The empirical illustration uses a real-time FRED-MD panel organized by data vintages. At vintage τ, the information set $\mathcal{F}_\tau$ contains the releases available at that vintage, the corresponding observation pattern, release-calendar information used in the implementation, and validation losses from earlier decisions whose validation values have already arrived. Candidate completions, states, and selected method classes are therefore vintage-feasible.

A panel cell is indexed by a series i and a reference date t. Let Y_{it} denote the validation value fixed by the empirical evaluation design; in this implementation it is the first value observed in a later vintage. Let r_{it} denote the first vintage at which that validation value is available. A cell belongs to the frontier at vintage τ if it is required for the empirical task and $r_{it} > \tau$. The decision unit is the triple $u = (i, t, \tau)$, because the same calendar cell may be completed at different vintages under different information sets and panel states.

The table below fixes the timing convention used throughout the empirical illustration. The validation date is measured in vintage-index time, not in calendar publication time. This is sufficient for the empirical exercise because validation values, candidate completions, state variables, and feedback availability are all defined relative to the sequence of vintages observed by the econometrician.

Table IA.1. Data layer

Item	Design choice
Data source	FRED-MD real-time vintages
Vintage range	1999-08–2024-12
Panel date range	1959-01–2024-11
Number of vintages	305
Number of panel series	101
Target variable	INDPRO
Decision unit	Frontier triple $u = (i, t, \tau)$
Validation value Y_{it}	First observed later-vintage value
Main frontier window	Six months

Notes: The table summarizes the real-time FRED-MD data layer used in the empirical illustration. The validation convention is vintage-index time: a decision unit is validated by the first later vintage in which the prespecified validation value is observed. The main six-month frontier window focuses the empirical exercise on current ragged-edge completion rather than historical backfill-type gaps.

The arrived validation archive accumulates mechanically as earlier frontier decisions are release-validated in later vintages. In the baseline window used in the empirical illustration, all 4,022 frontier decisions are eventually release-validated and the median reveal delay is one vintage, so the empirical exercise provides frequent delayed full-validation feedback while preserving the decision-date information constraint.

2 Frontier-Window Diagnostics

The main empirical illustration uses a six-month frontier window. This is a design choice, not a tuning outcome. The purpose is to focus on current ragged-edge completion rather than retrospective reconstruction of historical gaps. Wider windows are reported only as diagnostics: they show whether the release-validation sample changes materially when older incomplete cells are admitted.

For a frontier horizon H, define

$$\mathcal{U}^{Y}_{\tau,H} = \{u = (i,t,\tau) : (i,t) \in \mathcal{C}_\tau,\ r_{it} > \tau,\ 0 \le e(\tau) - t \le H\},$$

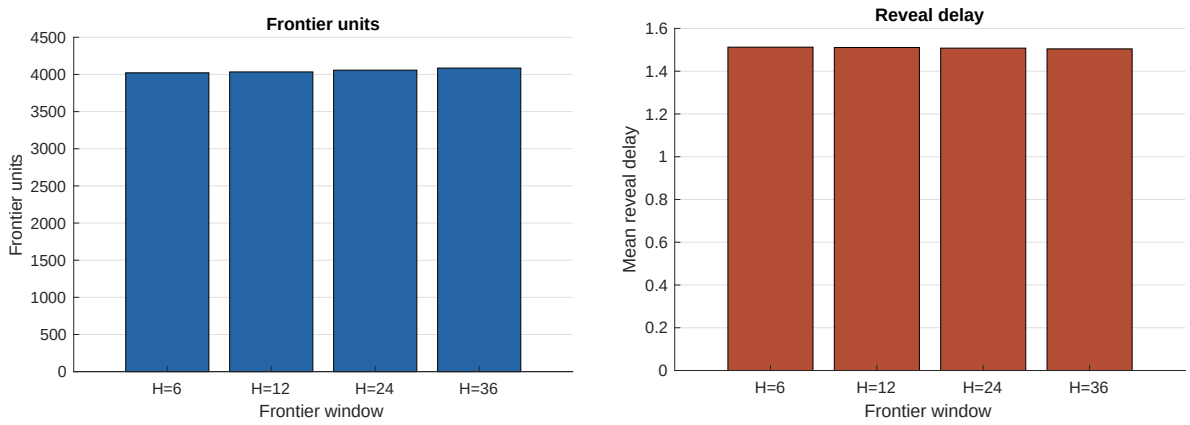
where $e(\tau)$ is the last panel reference date at vintage τ. The baseline choice is $H = 6$. The diagnostic exhibit compares $H = 6$ with wider windows.

Table IA.2. Frontier windows

Window	Frontier	Validated	Vintages	Median delay	Mean delay	Never validated	Avg. units/vintage
$H = 6$	4,022	4,022	304	1.0	1.51	0 (0.0%)	13.2
$H = 12$	4,034	4,034	304	1.0	1.51	0 (0.0%)	13.3
$H = 24$	4,058	4,058	304	1.0	1.51	0 (0.0%)	13.3
$H = 36$	4,086	4,086	304	1.0	1.50	0 (0.0%)	13.4

Notes: The table reports release-validation diagnostics for alternative frontier windows. Window length H is measured in monthly reference-date distance from the vintage endpoint $e(\tau)$. The baseline $H = 6$ window identifies the current ragged-edge completion block used in the main empirical illustration. Wider windows are diagnostic and may include older gaps, benchmark revisions, or backfill-like missingness. Reveal delays are measured in vintage steps among validated units.

The diagnostic also shows that widening the frontier window changes the empirical sample only marginally in this FRED-MD implementation. This supports the use of the six-month window as a transparent current-frontier convention rather than as a tuning choice that drives the release-validation results.



(a) Frontier units **(b)** Reveal delay

Figure IA.1. Frontier-window profiles

Notes: The figure reports how the frontier decision population and reveal-delay profile vary with the frontier window. The baseline window $H = 6$ is a design choice for current ragged-edge completion. Wider windows are retained as diagnostics because they can admit older historical gaps and backfill-like cells.

3 Method-Class Implementation

The empirical menu contains fixed vintage-feasible completion rules, grouped into method classes. The sequential selector is not an additional completion rule; it is the state-to-method rule that chooses among the fixed classes using the validation archive available at the decision vintage. Each class is documented at the level needed for replication: the vintage-feasible inputs, the implementation rule, fallback behavior, and the real-time restriction.

The descriptions are intentionally class-level rather than code-level. The objective is to record which information each method may use at the decision vintage and how failures of the preferred input are handled. Detailed numerical tolerances, convergence settings, and archived outputs belong to the replication package.

Table IA.3. Method classes

Method class	Vintage-feasible inputs	Implementation rule	Real-time restriction
Local-history	Own-series history observed in the decision vintage	Last available same-series observation before the reference date	Uses only observations available before validation at vintage τ
Cross-sectional pooling	Cross-section observed at the reference date in the decision vintage	Standardized cross-sectional mean at the reference date, back-transformed to the target-series scale	Uses only contemporaneous values observed at vintage τ
Common-component	Vintage panel observed by the decision vintage	Two-factor low-rank reconstruction after vintage standardization and mean initialization of missing cells	Factors are estimated only from the vintage-τ panel
Dynamic-filtering	Own-series one-sided dynamic history observed by the decision vintage	One-sided exponentially weighted local dynamic update	Filtering is one-sided and never uses future validation values

Notes: The table documents the fixed method classes used in the empirical release-validation and target-object layers. Fallback rules use only vintage-feasible same-series or panel means when the preferred input for a class is unavailable. All candidate completions are measurable with respect to the decision-vintage information set. The sequential selector maps states into these fixed method classes and is not an additional completion rule.

4 Release-Validation Diagnostics

This section documents the state construction and learning diagnostics behind the release-validation results. The main text reports that the common-component method is the strongest fixed release-validation benchmark and that the feasible selector remains close to it without improving on it in local release loss. The diagnostics below explain why this happens: most frontier decisions fall in states where the common-component rule is also the state oracle, and the only state favoring dynamic filtering is comparatively small.

Table IA.4. State counts

Operational state	State definition	Frontier units	Validated units	Share	First initialized vintage
gap0 medium	gap0; coverage medium	2,339	2,339	58.2%	2000-01
gap1–2 pooled	gap1–2; coverage pooled	874	874	21.7%	2000-02
gap0 low	gap0; coverage low	532	532	13.2%	2001-06
gap3–6 high	gap3–6; coverage high	277	277	6.9%	2001-06

Notes: The table reports the operational state partition used by the feasible release-validation selector. States are based on vintage-available gap and coverage information, with sparse raw cells pooled by the rule documented in the data archive. All 4,022 frontier units in the six-month window ($H = 6$) are release-validated in this sample. The first initialized vintage is the first vintage at which the state meets the prespecified archive threshold.

The state partition is intentionally coarse. A finer partition would create more state cells but would also increase initialization and estimation costs. The table shows that the release-validation sample is concentrated in the current-gap medium-coverage state, while the high-coverage longer-gap state is much smaller.

Table IA.5. State risks

State	Records	Local-history	Pooling	Common-component	Dynamic-filtering	State oracle class
gap0 medium	2,339	2.447	1.056	0.932	1.247	common-component
gap1–2 pooled	874	2.021	1.479	1.384	1.461	common-component
gap0 low	532	2.886	2.049	1.532	2.075	common-component
gap3–6 high	277	1.583	1.790	1.692	1.309	dynamic-filtering

Notes: Entries are mean squared local release-validation losses by operational state and fixed method class. Lower values indicate better local release performance. The state oracle class is infeasible and is reported only to summarize the amount of state-dependent heterogeneity available to a feasible selector.

The risk table summarizes the empirical mechanism. The common-component class is the state oracle in the three largest states. Dynamic filtering is best only in the smaller longer-gap high-coverage state. Hence the population value of state dependence is positive but limited, and a feasible selector must learn a small state-specific departure from an already strong fixed benchmark.

Table IA.6. Selector diagnostics

Diagnostic	Value	Interpretation
Initialization vintage	2000-01	First vintage satisfying the state archive threshold
Uninitialized target share	3.1%	Share of decisions using the default common-component rule
Oracle-static gain	0.026	Best fixed risk minus infeasible state-oracle risk
Selector-static gap	0.027	Feasible selector risk minus best fixed risk
Largest selector assignment share	95.0%	Largest method share assigned by the feasible selector
Selector assignment shares	local 0.0%; pooling 1.1%; common 95.0%; dynamic 3.9%	Assignment frequencies across decisions in the $H = 6$ window

Notes: The table connects the empirical release-validation result to the gain-cost decomposition. The selector uses only validation losses that arrived before each decision vintage. The common-component method is often close to the state oracle, so the estimated oracle-static gain is small and the feasible selector need not dominate the best fixed benchmark.

The selector diagnostics are consistent with the state-risk evidence. The feasible rule assigns the common-component method in most decisions and deviates from it only rarely. The selector-static gap is the finite-sample cost of learning a small amount of state dependence.

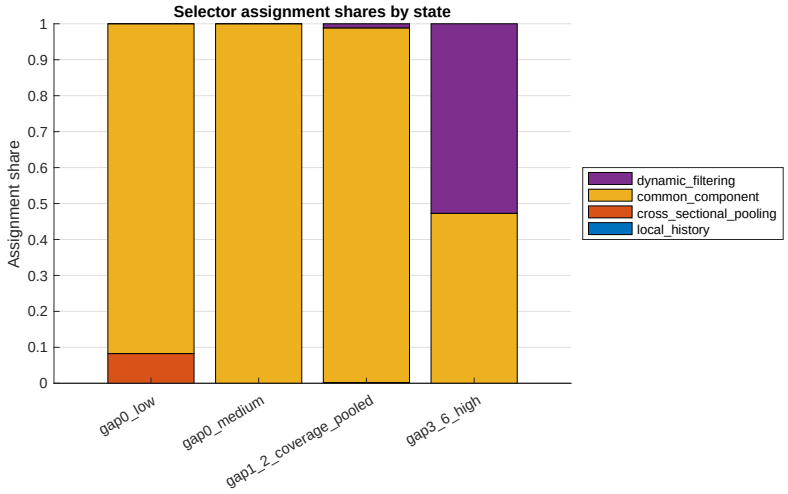


Figure IA.2. Selector shares

Notes: The figure reports assignment shares of the feasible release-validation selector by operational state. The selector chooses among fixed method classes and is not an additional completion rule. The assignment pattern illustrates why the selector remains close to the common-component benchmark when state-dependent gain is small.

5 Target-Object Design

The downstream target-object exercise evaluates each completed panel through the same target equation. This section records the timing and loss object for that exercise. Its purpose is not to introduce a new forecasting model, but to show how local release-validation rankings can be compared with a downstream loss computed from a

completed panel. For a fixed method class k, let $\widehat{Y}_{\tau}^{(k)}$ denote the panel completed at vintage τ. The corresponding prediction or estimate of the target object $Z_{\tau+h}$ is

$$\widehat{Z}_{\tau+h|\tau}^{(k)} = \Phi_{\tau,h}\left(\widehat{Y}_{\tau}^{(k)}, \mathcal{F}_{\tau}\right),$$

and the target loss is

$$L_{\tau}^{Z}(k) = L^{Z}\left(\widehat{Z}_{\tau+h|\tau}^{(k)}, Z_{\tau+h}\right).$$

The map $\Phi_{\tau,h}$ is vintage-available and common across completed panels. Thus the target-object comparison changes the completion method or selection rule, not the downstream equation. This convention isolates the contribution of the completion layer. Any difference in target-object loss is attributable to the completed panel supplied to the common downstream equation, not to changing the target model across methods.

Table IA.7. Target design

Item	Design choice
Target object	INDPRO
Horizon	One month
Forecast origins/evaluation points	304 monthly evaluation origins
Target equation	Factor-augmented AR(1) for INDPRO
Predictors	Constant, lagged INDPRO, first principal component of the completed panel
Completion variation	Fixed method class or feasible release-validation selection rule
Target realization timing	First later vintage in which the target reference date is observed

Notes: The table audits the target-object layer. The target is INDPRO and the horizon is one reference month ahead from the vintage endpoint. The same downstream equation is applied to every completed panel, so differences in target-object loss reflect the completion layer rather than changes in forecasting specification. Target reference dates are evaluated only when the corresponding INDPRO validation value appears in a later vintage.

Table IA.8. Target losses

Method or rule	Local RMSE	Target MSE	Target RMSE	Target MAE	Relative target MSE	Difference vs benchmark
Local-history	1.534	1.812	1.346	0.663	1.001	0.002
Cross-sectional pooling	1.153	1.792	1.338	0.664	0.990	-0.019
Common-component	1.078	1.810	1.345	0.665	1.000	0.000
Dynamic-filtering	1.186	1.795	1.340	0.663	0.992	-0.015
Best fixed release-validation benchmark	1.078	1.810	1.345	0.665	1.000	0.000
Sequential selector	1.091	1.810	1.345	0.665	1.000	0.000

Notes: The table reports target-object losses for the downstream INDPRO equation. Relative target MSE is normalized to the target MSE of the best fixed release-validation benchmark, common-component. Local RMSE is reported for comparison with the release-validation layer. The sequential selector is a decision rule, not an additional completion rule, and bootstrap inference for prespecified differences is reported separately.

Table IA.8 reports target-object losses together with local RMSE. The comparison shows that local release accuracy and downstream target-object loss are close but not identical criteria. Cross-sectional pooling and dynamic filtering slightly improve the downstream MSE relative to the local release-validation benchmark, but the differences are small and are summarized inferentially in Section 6.

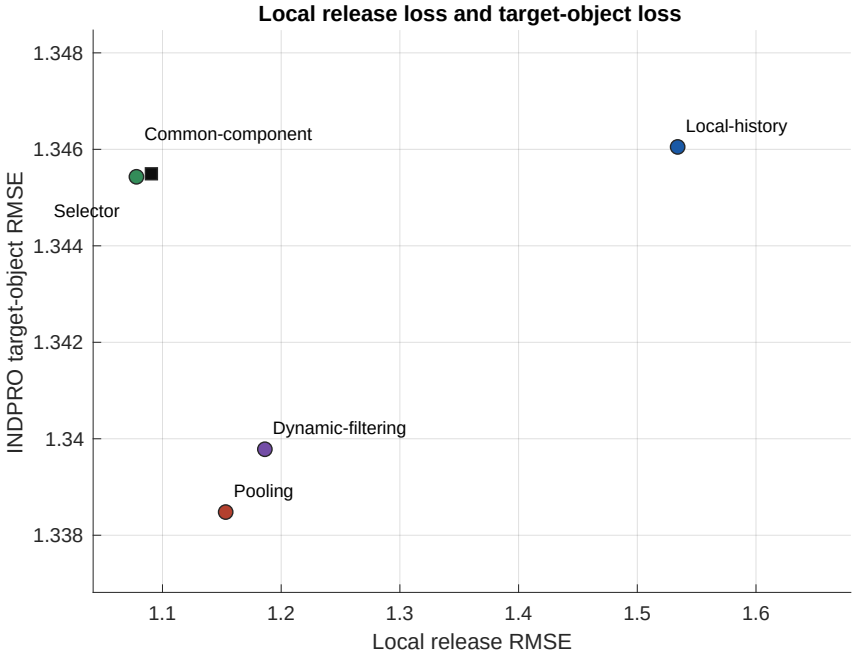


Figure IA.3. Local versus target loss

Notes: The figure plots local release RMSE against INDPRO target-object RMSE for the fixed method classes and the feasible selector. The downstream target equation is common across completed panels. The comparison illustrates that release-validation rankings and target-object rankings need not coincide exactly.

6 Bootstrap Inference

The bootstrap is an inferential layer for prespecified empirical comparisons. It is not part of the sequential selection rule and is not used to update method choices. The release-validation selector is constructed from validation losses that have arrived by the decision date; bootstrap calculations are applied only after the evaluation sample has been constructed. This separation is important. The bootstrap does not update the selection rule, choose states, select comparisons, or search for a better method. It summarizes the uncertainty in average loss differences generated by an already fixed real-time experiment.

The inferential target is limited by design. The bootstrap is used to describe sampling uncertainty in pairwise loss-difference averages, not to establish uniform domi-

nance of one method over the whole menu. This places the target-object exercise in the tradition of predictive-ability and forecast-comparison inference, including Diebold and Mariano (1995), West (1996), Clark and McCracken (2001), and Giacomini and White (2006). The real-time nature of the exercise follows the logic of vintage-data evaluation emphasized by Croushore and Stark (2001). The dependent-data resampling device follows the block-bootstrap tradition of Künsch (1989), Politis and Romano (1994), and Lahiri (2003).

There are two bootstrap problems. The first concerns local release-validation losses. The observations are frontier decisions $u = (i, t, \tau)$, and many frontier decisions share the same decision vintage, release environment, macro shock, or panel state. The relevant resampling unit is therefore a calendar block of decision vintages, with all frontier decisions inside the sampled block kept together. The second concerns target-object losses. The observations are forecast or nowcast origins τ, and the object is a time-ordered sequence of loss differences. This second problem is closest to Gonçalves et al. (2025), who develop a block-bootstrap approach to out-of-sample predictive-ability inference with real-time data and regular revisions. Our use of their logic is conservative: we resample blocks of forecast origins after the real-time evaluation sequence has been fixed, and we use the resulting distribution only to summarize uncertainty in prespecified target-object loss differences.

All bootstrap comparisons below are pairwise and prespecified. The reported p-values are descriptive two-sided bootstrap summaries, not post-selection evidence from searching over many methods. The baseline block length is six monthly origins. This choice is fixed before inspecting significance patterns, is not selected to optimize any reported comparison, and is intended to preserve short-run dependence in monthly loss differences. The same number of replications, $B^* = 999$, is used for all reported comparisons. In this sense, the bootstrap layer is closest to conditional predictive-ability inference: the realized selector assignments and the realized evaluation path are treated as fixed when forming the bootstrap distribution.

6.1 Bootstrap statistic and interpretation

This subsection records the statistic and interpretation behind the bootstrap calculations. They are used for prespecified empirical comparisons between completion methods or rules, not for the sequential learning algorithm itself. For a target-object comparison between two methods or rules a and b, let

$$d_\tau(a, b) = L_\tau^Z(a) - L_\tau^Z(b), \qquad \tau \in \mathcal{T}_Z,$$

where $\mathcal{T}_Z$ is the set of target-object evaluation origins. Write $T_Z = |\mathcal{T}_Z|$ and

$$\bar{d}(a,b) = T_Z^{-1} \sum_{\tau \in \mathcal{T}_Z} d_\tau(a,b).$$

Let

$$\bar{\mu}_d = T_Z^{-1} \sum_{\tau \in \mathcal{T}_Z} \mathbb{E}[d_\tau(a,b)].$$

The expectation is taken with respect to the underlying, potentially heterogeneous real-time data-generating process, so the mean loss difference may vary across evaluation origins. The first-order approximation concerns

$$\sqrt{T_Z}\{\bar{d}(a,b) - \bar{\mu}_d\}.$$

The bootstrap standard errors and intervals are computed from the conditional distribution of the centered statistic

$$\bar{d}^*(a,b) - \bar{d}(a,b),$$

where $\bar{d}^*(a,b)$ is obtained by the moving-block bootstrap: with block length ℓ_B, the overlapping blocks of ℓ_B consecutive evaluation origins, with starting points $s = 1, \ldots, T_Z - \ell_B + 1$, are drawn independently and uniformly with replacement, concatenated, and truncated to length T_Z, and $\bar{d}^*(a,b)$ is the average of the resampled loss differences. Validity in the present setting does not require stationarity, but it does require weak dependence, uniform moment bounds, and a non-degenerate limit. The following high-level condition is maintained for every prespecified comparison.

Assumption IA.1. *Let W_τ denote the origin-level observation used in a given comparison: $W_\tau = d_\tau(a,b)$ for target-object comparisons, and $W_\tau = (D_\tau(a,b), M_\tau)'$ for release-validation comparisons, with the sample size T_Z read as the number of decision vintages in the release layer.*

(a) $\{W_\tau\}$ is a possibly heterogeneous, weakly dependent array: strong mixing, or near-epoch dependent on a mixing array, with mixing or dependence coefficients of the sizes and uniformly bounded moments of order $2+\varepsilon$, $\varepsilon > 0$, required by moving-block theory for dependent heterogeneous arrays.

(b) The long-run covariance matrix of the normalized centered partial sums of $\{W_\tau\}$ converges to a finite limit that is positive along the contrast of interest.

(c) In the release layer, $\liminf_{T_Z \to \infty} T_Z^{-1} \sum_\tau \mathbb{E}[M_\tau] > 0$.

(d) The block length satisfies $\ell_B \to \infty$ *and* $\ell_B / T_Z \to 0$.

Proposition IA.1. *Suppose Assumption IA.1 holds. Then, as* $T_Z \to \infty$*:*

(i) $\sqrt{T_Z}\{\bar{d}(a,b) - \bar{\mu}_d\}$ *is asymptotically normal, and the moving-block bootstrap is consistent:*

$$\sup_{x \in \mathbb{R}} \left| \mathbb{P}^* \left[\sqrt{T_Z}\{\bar{d}^*(a,b) - \bar{d}(a,b)\} \le x \right] - \mathbb{P}\left[\sqrt{T_Z}\{\bar{d}(a,b) - \bar{\mu}_d\} \le x \right] \right| \to_p 0,$$

where $\mathbb{P}^*$ *denotes the bootstrap distribution conditional on the sample.*

(ii) The same conclusions hold for the release-validation ratio

$$\bar{\delta}^Y(a,b) = \frac{\sum_\tau D_\tau(a,b)}{\sum_\tau M_\tau}, \qquad \bar{\mu}_\delta = \frac{\sum_\tau \mathbb{E}[D_\tau(a,b)]}{\sum_\tau \mathbb{E}[M_\tau]},$$

and its bootstrap analogue, by the delta method applied to the vintage-level mean vector.

Proposition IA.1 assembles known results and claims no new bootstrap theory. Part (i) follows from central limit theory and moving-block consistency for dependent *heterogeneous* arrays: Gonçalves and White (2002) establish bootstrap consistency for means of near-epoch-dependent, heterogeneously distributed data, and Fitzenberger (1998) covers moving-block inference under heteroskedasticity and autocorrelation; Künsch (1989) and Lahiri (2003) provide the stationary background. The heterogeneous-array results, rather than stationary theory alone, are the relevant justification here because the evaluation window spans the 2008–2009 and 2020 episodes, so the loss-difference array is best treated as non-identically distributed. Part (ii) follows by applying part (i) to the vector $W_\tau = (D_\tau, M_\tau)'$ and the map $(x, y) \mapsto x/y$, which is continuously differentiable at the limit of the vintage-level means because the denominator is bounded away from zero by part (c); the joint block resampling of (D_τ, M_τ) is what makes this delta step valid, since it preserves the within-vintage dependence between loss sums and validation counts.

Interval and p-value constructions are fixed as follows. Confidence intervals are equal-tailed percentile intervals from the bootstrap distribution of the average; descriptive two-sided p-values are symmetric and computed from the centered bootstrap distribution, $p = (B^*)^{-1} \sum_{b=1}^{B^*} \mathbf{1}\{|\bar{d}_b^* - \bar{d}| \ge |\bar{d}|\}$. Because the two constructions answer slightly different questions, an interval endpoint near zero need not correspond to a small p-value, especially when the bootstrap distribution is asymmetric or highly concentrated. In the empirical implementation, the block length is fixed at six monthly

origins to match the short-run data-release cycle and is used only as a finite-sample uncertainty summary. The resulting intervals are not a new asymptotic result about optimal block-length choice or about the full sequential learning rule.

For release-validation comparisons, let V_τ^Y be the set of release-validated frontier decisions made at decision vintage τ. For a comparison between rules or method classes a and b, define

$$D_\tau(a,b) = \sum_{u \in V_\tau^Y} \delta_u^Y(a,b), \qquad M_\tau = |V_\tau^Y|.$$

Then the release-validation average can be written as the ratio

$$\bar{\delta}^Y(a,b) = \frac{\sum_\tau D_\tau(a,b)}{\sum_\tau M_\tau}.$$

In each bootstrap replication, blocks of decision vintages are resampled and the bootstrap analogue is

$$\bar{\delta}^{Y,*}(a,b) = \frac{\sum_{\tau \in \mathcal{B}^*} D_\tau(a,b)}{\sum_{\tau \in \mathcal{B}^*} M_\tau},$$

where $\mathcal{B}^*$ denotes the resampled multiset of decision vintages induced by the sampled blocks. This ratio form keeps all frontier decisions within a sampled vintage block together and preserves the empirical weighting induced by unequal numbers of frontier decisions across vintages. Because M_τ varies across decision vintages, the release-validation bootstrap resamples the vintage-level vector $(D_\tau(a,b), M_\tau)$ jointly rather than resampling normalized vintage averages. Assumption IA.1(c) and Proposition IA.1(ii) record the conditions under which this ratio scheme is asymptotically valid.

The distinction between conditional and unconditional inference is essential. Conditional on the realized decision path, the bootstrap describes uncertainty in the loss differences delivered by that path. It does not account for the additional variation that would arise if the entire sequential selection experiment were repeated from scratch and the selector followed a different path. One further distinction sharpens the scope. For comparisons between fixed method classes, no data-driven selection is involved at all: under delayed full validation, every class is scored on every validated unit, so the loss-difference sequence is a fixed function of the data and raises no post-selection concerns. Only the rows involving the feasible selector depend on the realized learning path, and for those the conditional interpretation is the operative one. The resulting intervals are therefore interpreted as finite-sample uncertainty summaries for prespecified loss differences, and for gain-style diagnostics when the relevant benchmark assignments are treated as fixed or sufficiently separated, rather than as a new asymptotic theory for

the selected method or for the full sequential learning path. For gain-style quantities involving minima, the bootstrap output is reported as a diagnostic uncertainty summary rather than as formal inference for a nonsmooth argmin functional. The selected method identity is an argmin object and is not bootstrapped here; its asymptotic behavior is governed by the margin conditions discussed in the main text. This limitation matches the role of the empirical exercise: the bootstrap supports cautious comparisons among prespecified rules and methods, while the theoretical learning results in the main text describe the broader gain-cost logic of sequential selection.

6.2 Release-validation differences

For two prespecified rules or method classes a and b, define

$$\delta_u^Y(a,b) = \ell_u^Y(a) - \ell_u^Y(b), \qquad \bar{\delta}^Y(a,b) = \frac{1}{|\mathcal{V}_Y|} \sum_{u \in \mathcal{V}_Y} \delta_u^Y(a,b),$$

where $\mathcal{V}_Y$ is the set of release-validated frontier decisions. Negative values favor a. The comparison is local: it concerns release-validation loss, not downstream target-object loss.

The release-validation bootstrap resamples blocks of consecutive decision vintages. For each sampled block, all release-validated frontier decisions whose decision vintages fall in the block are retained. This preserves the dependence induced by common vintage information, release-calendar clustering, repeated completions of related cells, and cross-sectional macro shocks. The feasible selector is not re-estimated inside each bootstrap sample. It is treated as the real-time rule produced by the original sequential selection experiment, and the bootstrap summarizes uncertainty in the resulting loss-difference sequence. Thus the release-validation bootstrap is conditional on the realized training and decision path of the selector. It does not approximate the sampling distribution that would arise from rerunning the full sequential selection experiment on new real-time panels.

Table IA.9. Release bootstrap

Comparison	Mean diff.	Bootstrap SE	95% CI	p-value	Block length	Replications
Selector vs best fixed release-validation benchmark	0.0273	0.0505	[−0.0532, 0.1449]	0.711	6	999
State oracle vs best fixed release-validation benchmark	-0.0263	0.0228	[−0.0737, 0.0107]	0.238	6	999

Notes: Negative values favor the first rule in the comparison. Loss differences use local release-validation losses on frontier units in the six-month ($H = 6$) window. The bootstrap resamples calendar blocks of decision vintages and preserves all frontier units inside sampled blocks. The selector comparison concerns the feasible real-time rule, conditional on the realized decision path. The state-oracle comparison is diagnostic and infeasible: it summarizes uncertainty around the estimated amount of state-dependent heterogeneity, not the performance of an implementable rule. Confidence intervals are equal-tailed percentile intervals; descriptive two-sided p-values are symmetric and computed from the centered bootstrap distribution (Section 6.1). Confidence-interval endpoints are rounded to four decimal places.

Table IA.9 reinforces the main release-validation interpretation. The feasible selector is close to the best fixed benchmark, but the estimated difference is small relative to bootstrap uncertainty. The infeasible state oracle improves on the best fixed benchmark on average, but its interval also includes zero. This is the empirical counterpart of the gain-cost logic: the state-dependent gain exists, but it is small in this FRED-MD frontier window.

6.3 Target-object differences

For two prespecified methods or rules k and m, define

$$d_\tau(k, m) = L_\tau^Z(k) - L_\tau^Z(m), \qquad \tau \in \mathcal{T}_Z,$$

where $\mathcal{T}_Z$ is the set of target-object evaluation origins. Negative values favor method k. The average difference is

$$\bar{d}(k, m) = \frac{1}{|\mathcal{T}_Z|} \sum_{\tau \in \mathcal{T}_Z} d_\tau(k, m).$$

The target-object comparison is the closest analogue to real-time forecast-comparison inference. The completed panel, the target equation, and the target realization are all indexed by the real-time evaluation origin. Gonçalves et al. (2025) show how block bootstrap methods can be used for predictive-ability inference when forecasts are constructed from real-time vintage data subject to regular revisions. Our implementation follows the same organizing principle: once the sequence of real-time target-object loss differences has been fixed, we resample blocks of consecutive evaluation origins to preserve serial dependence, vintage dependence, and revision-related dependence in the

loss-difference sequence.

The target-object bootstrap is not a bootstrap of the entire sequential selection experiment. In particular, it does not rerun the release-validation selector inside each resample. This is intentional. The object reported in Table IA.10 is the uncertainty around prespecified average target-object loss differences for the completed panels generated by the original real-time exercise. As in the release-validation layer, the approximation is conditional on the realized real-time path: the completed panels, target equation, selector decisions, forecast origins, and target realizations used to form the loss-difference sequence are fixed before resampling begins.

Table IA.10. Target bootstrap

Comparison	Mean diff.	Bootstrap SE	95% CI	p-value	Block length	Replications
Pooling vs common-component	-0.0187	0.0178	[−0.0599, 0.0033]	0.162	6	999
Dynamic-filtering vs common-component	-0.0152	0.0144	[−0.0478, 0.0020]	0.116	6	999
Selector vs common-component	0.0002	0.0002	[0.0000, 0.0005]	0.330	6	999
Local-history vs common-component	0.0017	0.0049	[−0.0051, 0.0131]	0.785	6	999

Notes: Negative values favor the first method or rule. Loss differences are computed on downstream INDPRO target-object squared loss, not on local release loss. The moving block bootstrap resamples consecutive evaluation origins and is used only to summarize uncertainty for prespecified comparisons against the common-component benchmark. Confidence intervals are equal-tailed percentile intervals; descriptive two-sided p-values are symmetric, computed from the centered bootstrap distribution (Section 6.1), and not adjusted for searching over multiple comparisons. The selector coincides with the common-component completion at almost all origins, so its loss-difference sequence is zero for most τ; the corresponding bootstrap distribution is highly concentrated near zero, and that row should be read as documenting near-equivalence rather than as a precise interval. Confidence-interval endpoints are rounded to four decimal places.

Table IA.10 supports a cautious reading of the downstream exercise. Pooling and dynamic filtering have lower average target-object loss than the common-component benchmark, but the bootstrap intervals include zero. The selector is essentially indistinguishable from the common-component method on the target-object criterion. The table therefore documents the validation-target distinction without turning the empirical illustration into a claim of target-object dominance.

Algorithm IA.1 Target-object block bootstrap

1: Fix a prespecified pair (k, m) and compute $d_\tau(k, m)$ over target-object evaluation origins.
2: Choose a block length ℓ_B and number of bootstrap replications B^* before inspecting significance patterns.
3: **for** $b = 1, \ldots, B^*$ **do**
4: Draw blocks of consecutive evaluation origins until the bootstrap sequence has length at least $|\mathcal{T}_Z|$.
5: Truncate to length $|\mathcal{T}_Z|$ and compute the bootstrap average loss difference.
6: **end for**
7: Report the point estimate, bootstrap standard error, confidence interval, and descriptive two-sided p-value.

7 Additional Monte Carlo Output

This section collects simulation material that is useful for auditability but too detailed for the main text. The main paper reports the core Monte Carlo evidence and discusses the gain-cost mechanism. The appendix records the calibration, bridge diagnostics, scaling exercises, application-size regimes, and bridge-strength diagnostics so that the simulation claims in the main text can be checked without interrupting the main narrative.

Table IA.11. Monte Carlo calibration

Design component	Calibration
Baseline size	$R = 1000$, $N_0 = 60$, $T_0 = 250$.
Panel DGP	$Y_{it} = \lambda_i f_t + u_{it}$, $u_{it} = \rho_i u_{i,t-1} + \sigma_i \varepsilon_{it}$; heterogeneity in $(\lambda_i, \rho_i, \sigma_i)$ generates state-dependent rankings.
States	$G_u \in \{F, P, L\}$: factor-dominant, persistence-dominant, and noisy/local cells; G_u is observed at the decision date.
Method menu	$\mathcal{K} = \{\text{Local}, \text{Pool}, \text{Factor}\}$: own-lag persistence, lagged cross-sectional pooling, and one-step factor forecast.
Feasibility	$\widehat{Y}^{(k)}_{it\|\tau} = q_k(u; \mathcal{F}_\tau)$; the validation value Y_{it} enters only when validation loss is computed.
Initialization	Full validation: $m = 2$ validated records per state, equivalently per state-method cell because all methods are scored on each validated unit; selected-arm feedback: $m = 2$ observed records per state-method cell. Sparse-feedback design: $m_{S2} = 20$.
Delays	$(\bar{D}, D_{\max}) = (2, 8)$ in S0, S1, S4, S5, and S6; $(22, 70)$ in S2; $(3, 12)$ in S3.
Feedback	Full validation: $\mathcal{K}^V = \mathcal{K}$; selected-arm feedback in S6: $\mathcal{K}^V = \{A_u\}$ with exploration probability $\epsilon = 0.12$.
S4 transport	$p^V(H) \neq p^T(H)$; correction reweights by observed subtype H.
S5 target object	Local-trained rule is learned from local validation and evaluated on target-object loss; the target-trained rule is infeasible.
Scaling grids	$T \in \{50, 100, 250, 500, 1000, 2000\}$ at $N = 60$; $N \in \{30, 60, 120, 240, 500, 1000\}$ at $T = 250$.
Application regimes	Stylized macro, finance, micro, and completed-panel dimensions mirror the cases in the main text.

Notes: The table records the notation and calibration choices needed to reproduce the Monte Carlo design. The basic panel equation and method menu are common across scenarios. Scenario-specific settings change validation timing, feedback observability, separation of state rankings, or the validation-target relation.

Table IA.12. Bridge diagnostics

Scenario	Metric	Mean	MCSE	Median	p10	p90
S3	Cell/block rank agreement	1.000	0.0000	1.000	1.000	1.000
S4	Unadjusted rule target risk	1.426	0.004	1.435	1.244	1.595
S4	Transport-corrected rule target risk	1.162	0.004	1.161	1.019	1.304
S4	Transport risk gain	0.264	0.004	0.266	0.104	0.417
S4	Rank agreement: unadjusted vs target	0.630	0.005	0.667	0.333	0.667
S4	Rank agreement: corrected vs target	0.867	0.005	1.000	0.667	1.000
S5	Local-trained rule target risk	0.706	0.003	0.699	0.582	0.837
S5	Infeasible target-trained rule risk	0.304	0.001	0.301	0.258	0.352
S5	Target-object deterioration	0.402	0.002	0.395	0.320	0.494
S5	Local-target rank agreement	0.341	0.002	0.333	0.333	0.333
S5	Local-target divergence indicator	0.659	0.002	0.667	0.667	0.667

Notes: S3 is an aligned aggregate-validation design. S4 concerns validation-population shift and transport correction. S5 concerns target-object evaluation. Local-trained rule means learned from local validation and evaluated on target-object loss. The target-trained rule is infeasible and is used only as a benchmark. Monte Carlo standard errors are reported in the adjacent MCSE column.

The bridge diagnostics separate aligned validation from validation-target mismatch. S3 confirms that changing the loss-bearing object from cells to blocks need not create a problem when the block loss is also the target criterion. S4 and S5 show the opposite case: when validation and target populations or criteria differ, the local validation ranking can be misleading unless an explicit bridge or transport correction is available.

Table IA.13. T-scaling

Scenario	Metric	Stat.	$T = 50$	$T = 100$	$T = 250$	$T = 500$	$T = 1000$	$T = 2000$
S1	G	Mean	0.292	0.301	0.306	0.305	0.309	0.310
S1	G	MCSE	0.002	0.002	0.002	0.002	0.002	0.002
S1	Regret	Mean	0.029	0.014	0.005	0.002	0.001	0.001
S1	Regret	MCSE	0.0006	0.0003	0.0001	< 0.001	< 0.001	< 0.001
S1	Recovered	Mean	0.893	0.952	0.982	0.991	0.996	0.998
S1	Recovered	MCSE	0.002	0.001	0.0004	0.0002	0.0001	< 0.001
S1	Init. share	Mean	0.025	0.012	0.005	0.002	0.001	0.001
S1	Archive size	Mean	734.9	1554.3	4015.4	8079.6	16036.3	32408.7
S1	Sparsity	Mean	0.187	0.173	0.170	0.174	0.187	0.180
S1	Oracle match	Mean	0.945	0.973	0.990	0.995	0.998	0.999
S2	G	Mean	0.238	0.248	0.254	0.260	0.260	0.265
S2	Regret	Mean	0.082	0.038	0.017	0.008	0.004	0.002
S2	Recovered	Mean	0.609	0.831	0.928	0.969	0.986	0.994
S2	Init. share	Mean	0.141	0.069	0.027	0.014	0.007	0.003
S2	Archive size	Mean	87.1	226.4	676.2	1459.3	2952.8	5967.0
S2	Sparsity	Mean	0.807	0.819	0.818	0.813	0.816	0.814
S2	Oracle match	Mean	0.773	0.823	0.886	0.939	0.972	0.990
S6	G	Mean	0.292	0.300	0.305	0.307	0.310	0.311
S6	Regret	Mean	0.149	0.126	0.101	0.092	0.083	0.078
S6	Recovered	Mean	0.460	0.564	0.657	0.690	0.726	0.743
S6	Init. share	Mean	0.040	0.020	0.008	0.004	0.002	0.001
S6	Archive size	Mean	39.5	71.5	168.6	330.0	651.3	1300.3
S6	Sparsity	Mean	0.527	0.465	0.403	0.375	0.348	0.333
S6	Oracle match	Mean	0.716	0.750	0.819	0.846	0.880	0.898

Notes: All entries use 1,000 replications. The table varies T at $N = 60$. For compactness, the table reports means for the main diagnostics and reports MCSE rows only for the headline S1 quantities. Recovered is the recovered gain share; Regret is relative to the state-dependent oracle; archive size is the terminal number of arrived validation records. Increasing T expands the validation archive over decision time and is especially important when validation is delayed or selected-arm feedback makes observed state-method losses sparse.

Table IA.14. N-scaling

Scenario	Metric	Stat.	$N = 30$	$N = 60$	$N = 120$	$N = 240$	$N = 500$	$N = 1000$
S1	G	Mean	0.302	0.307	0.305	0.306	0.306	0.307
S1	Regret	Mean	0.007	0.006	0.004	0.004	0.004	0.003
S1	Recovered	Mean	0.975	0.981	0.985	0.987	0.988	0.989
S1	Init. share	Mean	0.006	0.005	0.004	0.004	0.004	0.004
S1	Archive size	Mean	1829.8	4008.2	8489.9	17765.3	38224.5	77969.4
S1	Sparsity	Mean	0.235	0.170	0.128	0.092	0.063	0.045
S1	Oracle match	Mean	0.985	0.989	0.993	0.993	0.994	0.995
S2	G	Mean	0.240	0.254	0.263	0.269	0.268	0.271
S2	Regret	Mean	0.021	0.016	0.013	0.012	0.011	0.010
S2	Recovered	Mean	0.905	0.931	0.948	0.955	0.958	0.962
S2	Init. share	Mean	0.039	0.027	0.020	0.014	0.011	0.008
S2	Archive size	Mean	395.6	692.7	1369.1	2723.5	5604.5	11326.2
S2	Sparsity	Mean	0.776	0.813	0.820	0.826	0.828	0.827
S2	Oracle match	Mean	0.881	0.895	0.909	0.903	0.901	0.911
S3	G	Mean	0.114	0.114	0.115	0.115	0.115	0.115
S3	Regret	Mean	0.001	0.001	0.001	0.001	0.001	0.001
S3	Recovered	Mean	0.990	0.992	0.994	0.994	0.994	0.994
S3	Init. share	Mean	0.006	0.005	0.005	0.004	0.004	0.004
S3	Archive size	Mean	1786.9	3972.3	8525.4	17658.3	38042.4	77709.6
S3	Sparsity	Mean	0.249	0.179	0.123	0.097	0.065	0.045
S3	Oracle match	Mean	0.993	0.994	0.995	0.995	0.995	0.996

Notes: All entries use 1,000 replications. The table varies N at $T = 250$. For compactness, the table reports means for the main diagnostics. Recovered is the recovered gain share; Regret is relative to the state-dependent oracle; archive size is the terminal number of arrived validation records. Increasing N expands the cross-sectional dimension and stabilizes state-method feedback counts, but does not replace a long decision horizon.

Table IA.15. Application regimes

Application	Scenario	N	T	Recovered	Regret	Archive size	Bridge/target
Macro	S1	60	250	0.982	0.005	3973.9	–
Macro	S2	60	250	0.935	0.016	696.8	–
Macro	S1	60	500	0.991	0.003	8061.8	–
Macro	S2	60	500	0.969	0.008	1393.7	–
Macro	S1	60	1000	0.996	0.001	16185.6	–
Macro	S2	60	1000	0.986	0.004	3061.3	–
Macro	S1	120	250	0.986	0.004	8506.7	–
Macro	S2	120	250	0.943	0.014	1343.0	–
Macro	S1	120	500	0.993	0.002	17265.5	–
Macro	S2	120	500	0.975	0.007	2844.9	–
Finance	S3	250	1000	0.999	0.000	75202.3	–
Finance	S6	250	1000	0.733	0.081	3016.0	–
Finance	S3	500	2000	0.999	0.000	310447.4	–
Finance	S6	500	2000	0.748	0.077	12436.4	–
Micro	S4	500	20	0.567	0.122	534.7	0.216
Micro	S4	500	100	0.920	0.025	3303.3	0.295
Micro	S4	2000	100	0.947	0.017	13154.3	0.306
Completed	S5	60	250	–	0.407	4016.3	0.403
Completed	S5	120	500	–	0.397	17225.8	0.395
Completed	S5	240	500	–	0.398	35822.3	0.396

Notes: The table reports representative application-size regimes. These are scale calibrations for the Monte Carlo designs, not additional empirical applications. Macro regimes use delayed full-validation scenarios S1 and S2. Finance regimes use aligned aggregate validation (S3) and selected-arm feedback (S6). Micro regimes use S4; the bridge/target column reports the transport diagnostic. Completed-panel regimes use S5; the bridge/target column reports target-object deterioration. The full simulation archive contains the complete grid and MCSE columns.

Table IA.16. Bridge strength

Scenario	Strength	Diagnostic	Mean	MCSE	Median	p90
S4	0.00	Transport risk gain	-0.000	< 0.001	0.000	0.000
S4	0.50	Transport risk gain	-0.000	0.0002	-0.000	0.005
S4	1.00	Transport risk gain	0.256	0.004	0.261	0.409
S4	1.50	Transport risk gain	0.335	0.006	0.320	0.593
S4	2.00	Transport risk gain	0.389	0.008	0.369	0.710
S5	0.00	Target-object deterioration	0.190	0.002	0.184	0.258
S5	0.50	Target-object deterioration	0.298	0.002	0.297	0.374
S5	1.00	Target-object deterioration	0.402	0.002	0.395	0.494
S5	1.50	Target-object deterioration	0.503	0.003	0.502	0.606
S5	2.00	Target-object deterioration	0.603	0.003	0.597	0.730

Notes: S4 varies validation-target mismatch strength and reports the risk improvement from transport correction. S5 varies target-object divergence strength and reports the excess target-object risk of the local-trained rule relative to the infeasible target-trained benchmark. All entries use 1,000 replications.

The bridge-strength table makes the identification message explicit. In S4, transport correction matters only when validation and target compositions diverge. In S5, target-object deterioration grows with the strength of the local-target divergence. These are not learning failures: they are consequences of evaluating a rule under a target criterion that is not aligned with the validation criterion used to learn it.

References

Clark, T. E., McCracken, M. W., 2001. Tests of equal forecast accuracy and encompassing for nested models. Journal of Econometrics 105 (1), 85–110.

Croushore, D., Stark, T., 2001. A real-time data set for macroeconomists. Journal of Econometrics 105 (1), 111–130.

Diebold, F. X., Mariano, R. S., 1995. Comparing predictive accuracy. Journal of Business & Economic Statistics 13, 253–263.

Fitzenberger, B., 1998. The moving blocks bootstrap and robust inference for linear least squares and quantile regressions. Journal of Econometrics 82 (2), 235–287.

Giacomini, R., White, H., 2006. Tests of conditional predictive ability. Econometrica 74, 1545–1578.

Gonçalves, S., McCracken, M. W., Yao, Y., 2025. Bootstrapping out-of-sample predictability tests with real-time data. Journal of Econometrics 247, 105916.

Gonçalves, S., White, H., 2002. The bootstrap of the mean for dependent heterogeneous arrays. Econometric Theory 18 (6), 1367–1384.

Künsch, H. R., 1989. The jackknife and the bootstrap for general stationary observations. Annals of Statistics 17 (3), 1217–1241.

Lahiri, S. N., 2003. Resampling Methods for Dependent Data. Springer, New York.

Politis, D. N., Romano, J. P., 1994. The stationary bootstrap. Journal of the American Statistical Association 89 (428), 1303–1313.

West, K. D., 1996. Asymptotic inference about predictive ability. Econometrica 64 (5), 1067–1084.